

Teacher-Distilled Lightweight Graph Learning for Real-Time Dynamic Job Shop Scheduling

ZHONGKUAN MA
Northeast Forestry University
2024212760@nefu.edu.cn

Highlights

Teacher-distilled lightweight graph policy for real-time job shop dispatch.

Variable-size heterogeneous graph transfers across shop sizes without padding or retraining.

Beats SPT on 49 of 49 official JSPLIB instances and the best-of-five oracle on 33 of 49.

Reduces mean known-best gap from 56.9% to 34.0% via L2D teacher distillation.

Robust to dynamic job arrivals and machine failures in simulated scenarios.

Abstract

Job shop scheduling (JSS) is a fundamental combinatorial optimization problem in real-time manufacturing systems, where dynamic job arrivals, machine failures, and production-line reconfigurations require decisions in milliseconds. Dispatching rules such as Shortest Processing Time (SPT) remain the industrial standard because they are fast and interpretable, yet no single rule performs consistently across diverse shop configurations and disturbance patterns. Exact solvers provide optimal schedules but are computationally prohibitive for real-time control. Learning-based dispatch has emerged as a promising middle ground, but most methods rely on slow and unstable reinforcement learning, assume fixed shop topologies, and rarely generalize to dynamic or failure-prone environments.

This paper proposes teacher-distilled lightweight graph learning: a lightweight heterogeneous graph policy first trained on CP-SAT optimal trajectories for small shops, then fine-tuned on schedules generated by the official open-source L2D method [20] on 30 Taillard instances. The policy explicitly encodes both job and machine nodes, enabling joint reasoning about job priorities and machine contention. A variable-size heterogeneous graph is constructed directly from the current instance, so a single trained policy can be applied to unseen shop sizes without fixed padding or retraining. Across 50 held-out 6 x 5 instances, the policy improves makespan by 9.1% over SPT (129.4 vs. 142.4; $p < 0.0001$, 41/50 wins) and comes within 3.4% of the best-of-five oracle. On 49 official JSPLIB instances, the distilled policy improves over SPT on all 49 instances with a mean improvement of 25.2%, beats the best-of-five rule oracle on 33 of 49 instances, and reduces its mean known-best gap from 56.9% to 34.0%. On 37 official instances with matching released L2D checkpoints, the lightweight student remains on average 6.9% behind the published L2D teacher under (student - teacher) / teacher, winning 7 of 37. In dynamic arrival and machine failure scenarios, it outperforms SPT by 8.7% ($p = 0.021$) and 9.6% ($p = 0.033$). Under CP-SAT supervision on 60 generated 10x10 instances, the heterogeneous representation adds 3.3% over a homogeneous baseline; under L2D distillation on the official suite, the encoder effect is smaller, at 1.14%.

Our contributions are: (1) a teacher-distilled lightweight graph policy that approximates a strong published L2D dispatcher with a small variable-size model; (2) a variable-size heterogeneous graph construction that supports zero-shot topology transfer without padding or retraining; (3) stable transfer from static training to dynamic and failure-prone scheduling environments; (4) reproducible evaluation on 49 official JSPLIB instances with rule-relative improvements, known-best gaps, oracle comparisons, and teacher-student comparisons; and (5) a 42,178-parameter open implementation with 2.4-3.2 ms per CPU decision, released teacher schedules, training data, and scripts.

Keywords: job shop scheduling; heterogeneous graph neural network; teacher distillation; learning to dispatch; dispatching rules; topology generalization; dynamic scheduling; manufacturing optimization

1. Introduction

Manufacturing scheduling determines how production jobs are sequenced on shared machines to satisfy delivery requirements while minimizing operational costs. In modern flexible manufacturing systems, the scheduling problem is rarely static: customer orders arrive stochastically, machine breakdowns interrupt production, and product mix changes over time. These dynamic characteristics make the scheduling decision both time-critical and highly uncertain. The economic stakes are substantial: inefficient schedules translate directly into longer lead times, higher work-in-process inventory, lower machine utilization, and delayed deliveries.

The job shop scheduling problem (JSS), in which each job has an ordered sequence of operations requiring specific machines, is NP-hard in the strong sense. Exact methods, including branch-and-bound, mixed-integer programming, and constraint programming, can solve small instances optimally but face exponential growth in computational effort as problem size increases. For medium and large instances, exact solvers often require minutes to hours of computation, which is incompatible with the real-time decision cadence of shop-floor operations.

In industrial practice, dispatching rules dominate because they are simple, fast, and interpretable. Rules such as Shortest Processing Time (SPT), Longest Processing Time (LPT), Most Work Remaining (MWKR), Least Work Remaining (LWKR), and First-In-First-Out (FIFO) make decisions in microseconds using only local information. However, no single rule dominates across all instances, machine configurations, and objective functions. SPT tends to minimize mean flow time but can starve long jobs; MWKR prioritizes jobs with substantial remaining work but may delay short jobs; and FIFO ignores processing time information entirely.

This observation motivates an adaptive approach: instead of committing to a single rule, the scheduler should select the most appropriate decision at each state, based on the global structure of the problem. The machine learning community has recently made notable progress in this direction through learning-to-dispatch (L2D), in which a neural network is trained to select operations or rules based on learned representations of the scheduling state. Graph Neural Networks (GNNs) are particularly suited to this task because job shop instances naturally admit a graph representation.

Despite this progress, several gaps remain. First, most learning-based schedulers are trained with reinforcement learning, which requires careful reward shaping, exploration strategies, and large replay buffers, and can be slow or unstable to converge. The supervised-learning alternative, based on expert trajectories, has been underexplored; when used, it often relies on heuristic or approximate labels rather than proven optimal solutions. Second, most methods assume a fixed shop topology and must be retrained when the production line is reconfigured, which is impractical in reconfigurable manufacturing. Third, few studies explicitly model machine-level information through heterogeneous graph architectures, even though machine contention is the central constraint in JSS. Fourth, generalization from static training to dynamic arrivals, machine failures, and diverse benchmark families remains insufficiently validated.

This paper addresses these gaps through teacher-distilled lightweight graph learning. First, we use CP-SAT optimal trajectories on small instances to initialize a stable base dispatcher without unstable RL training. Second, we fine-tune the policy on schedules generated by the official open-source L2D method on 30 Taillard instances, transferring a strong published dispatcher into a lightweight student, and then compare it with released L2D checkpoints on all matching official sizes. Third, we construct a variable-size heterogeneous graph directly from the current instance, enabling zero-shot topology generalization without padding or retraining. Fourth, we validate across static, dynamic, failure-prone, and multiple official benchmark families, establishing statistical significance and practical robustness.

Our contributions are summarized as follows:

1. We propose a teacher-distilled lightweight graph policy for job shop scheduling that outperforms SPT on all 49 official JSPLIB instances and the best-of-five oracle on 33 of 49 instances.
2. We construct a variable-size heterogeneous graph directly from the current instance, allowing a single trained policy to transfer to previously unseen shop sizes without padding or retraining.

3. We demonstrate stable transfer to dynamic job arrivals and machine failures, outperforming SPT in both scenarios.

4. We provide extensive statistical evaluation on generated and official JSPLIB instances, including known-best gaps that make the remaining optimality gap explicit.

5. We release an open implementation with the teacher schedules, distilled checkpoint, training data, training scripts, and raw results, and show that the distilled policy runs in 2.4-3.2 ms per decision on CPU with only 42,178 parameters.

The remainder of this paper is organized as follows. Section 2 reviews related work. Section 3 describes the problem formulation and the lightweight graph policy. Section 4 presents the experimental setup and results. Section 5 discusses implications and limitations. Section 6 concludes.

2. Related Work

2.1 Classical Job Shop Scheduling Methods

The job shop scheduling problem has been studied extensively since the 1950s. [3] proposed the active schedule enumeration algorithm. [1] developed branch-and-bound methods for JSS, solving the famous 10 x 10 instance. Later, constraint programming solvers such as Google OR-Tools CP-SAT integrated propagation, search, and decomposition techniques to solve JSS instances of practical size. These exact methods provide optimality guarantees but scale poorly, limiting real-time applicability.

Metaheuristic methods, including genetic algorithms, simulated annealing, tabu search, and particle swarm optimization, offer a compromise between solution quality and computational cost. [11] introduced the tabu search algorithm i-TSAB. However, metaheuristics typically require substantial computation time and parameter tuning, making them less suitable for high-frequency scheduling decisions.

Dispatching rules have been the workhorse of industrial scheduling because they require minimal computation. [12] catalogued over 100 rules. The key insight is that rule performance is instance-dependent: no rule is universally dominant, motivating adaptive rule-selection approaches.

2.2 Learning-based Scheduling

Recent advances in deep learning have revitalized machine learning approaches to scheduling. [20] introduced L2D, a GNN-based policy that learns to dispatch operations from expert trajectories on the classic Taillard benchmark. Subsequent work extended L2D with attention-based architectures [13], reinforcement learning with reward shaping [18], hierarchical policies for flexible job shops [16], and graph isomorphism networks [6].

Reinforcement learning approaches have also gained traction. [21] formulated dynamic scheduling as a Markov decision process and trained PPO-based agents with GNN state encoders. [9] combined imitation learning and RL in a two-stage framework. These methods demonstrate the potential of learning-based scheduling, but they typically assume a fixed shop topology and require substantial training data.

2.3 Topology Generalization in Scheduling

A limitation of many learning-based scheduling methods is their dependence on a fixed problem topology. Most prior work addresses this by retraining the policy for each topology, which is impractical in dynamic manufacturing environments. Graph-based methods offer a natural mechanism because GNNs can process graphs of varying sizes. [13] used attention pooling for variable-size inputs; [6] demonstrated robustness to instance permutation. In this work, we use a variable-size heterogeneous graph whose node count follows the current instance, so topology transfer is obtained without padding or masking.

2.4 Heterogeneous Graph Networks and Flexible Job Shop Scheduling

Recent work has moved toward heterogeneous graph representations that explicitly encode machines, operations, and their interactions. [7] developed deep RL with heterogeneous GNNs for FJSP. [19] proposed meta-path-based heterogeneous graph networks. [17] handled machine breakdowns and material handling

constraints. [10] formulated distributed FJSP as multi-agent RL. [22] integrated Monte Carlo tree search with graph relational attention. Our paper contributes by combining teacher distillation, heterogeneous graphs, and zero-shot topology generalization.

2.5 Graph Neural Networks for Combinatorial Optimization

Beyond scheduling, GNNs have been applied to the traveling salesman problem [5], maximum independent set [8], and satisfiability [15]. These methods leverage the permutation equivariance of graph-structured data. Our work builds on this foundation with a focus on heterogeneous representations and topology generalization.

2.6 Comparison of Related Learning-based Scheduling Methods

Method	Graph type	Supervision	Topology transfer	Dynamic	FJSP
[20] L2D	Homogeneous	Expert	No	No	No
[13]	Attention	RL	Partial	No	Yes
[16]	Homogeneous	RL	No	No	Yes
[6]	Isomorphism	Expert	Partial	No	No
[17]	Heterogeneous	RL	No	Yes	Yes
[7]	Heterogeneous	RL	No	No	Yes
[19]	Meta-path	RL	No	No	Yes
[22]	Relational attn	MCTS+RL	No	Yes	No
Ours	Heterogeneous	CP-SAT + L2D teacher	Yes (zero-shot)	Yes	No (future)

Table 1: Table 0. Comparison of learning-based scheduling methods across graph type, supervision, topology transfer, dynamic capability, and FJSP support.

Table 0 positions our contribution: we combine heterogeneous graph representation, teacher distillation from a published L2D policy, zero-shot topology transfer, and dynamic/failure robustness in a single lightweight framework.

3. Method

3.1 Problem Formulation

A job shop scheduling problem instance is defined by a set of jobs $J = \{1, \dots, n\}$ and a set of machines $M = \{1, \dots, m\}$. Each job j has an ordered sequence of operations $O_j = (o_{\{j,1\}}, \dots, o_{\{j,k_j\}})$, where each operation $o_{\{j,k\}}$ requires a specific machine $\mu(o_{\{j,k\}})$ for a processing time $p_{\{j,k\}} \geq 0$. The scheduling decision assigns a start time $s_{\{j,k\}}$ to each operation, subject to: (1) job precedence constraints and (2) machine capacity constraints. The objective is to minimize the makespan $C_{\max}$.

JSS is NP-hard in the strong sense. This hardness motivates heuristic, metaheuristic, and learning-based approaches that trade optimality guarantees for computational efficiency.

3.2 Heterogeneous Graph Representation

We represent the scheduling state as a heterogeneous graph $G = (V, E)$ with two node types: job nodes and machine nodes. Job nodes encode the current operation duration, remaining work, total work, and progress for each job. Machine nodes encode normalized machine load and contention. Three edge types connect the graph: job-job edges (full connectivity), machine-machine edges (full connectivity), and job-machine edges connecting each job to the machine required by its current operation. This heterogeneous representation allows the GNN to reason jointly about job priorities and machine availability.

3.3 Heterogeneous Graph Attention Network Policy

The policy is a two-layer heterogeneous GAT. Type-specific linear projections embed job and machine features into a shared hidden space. Attention coefficients are computed over all edges, with softmax normalization per destination node. After message passing, job embeddings are combined with a machine-context vector (mean pooling of machine embeddings) to produce action logits. A policy head selects the available job with the highest score.

Formally, each GAT layer computes: $h_i^{(l+1)} = \text{ReLU}(\sum_{j \in N(i)} \alpha_{ij}^{(l)} W_{\text{type}(j)}^{(l)} h_j^{(l)})$, where type-specific projection matrices W handle node-type heterogeneity. The attention coefficient α_{ij} is computed as softmax over neighbors of $\text{LeakyReLU}(a^T [W h_i \parallel W h_j])$.

For topology generalization, the graph is constructed from the current instance size. Job and machine nodes are created only for the active shop, so a single trained policy can be applied to unseen numbers of jobs and machines without padding or retraining.

3.4 Expert Trajectory Generation via CP-SAT and L2D Teacher

We use a two-stage expert generation process. First, Google OR-Tools CP-SAT solves small 6x5 training instances to optimality, and expert actions are derived from optimal start times. Second, the released open-source L2D model [20] generates full schedules on ta01-ta30; at each state, the teacher action is the available job whose next operation starts earliest in the L2D schedule. The second stage transfers strong large-instance dispatch behavior into the lightweight heterogeneous student.

3.5 Training Objective

The policy is trained with cross-entropy loss on expert actions: $L = -(1/N) \sum \log \pi(a_i - s_i)$. We use Adam (lr=1e-3), batch size 32-64, 25-50 epochs, gradient clipping norm 1.0. Algorithm 1 summarizes the training procedure.

Algorithm 1: two-stage teacher-distilled HGNN policy training

Input: CP-SAT small-instance trajectories, L2D teacher schedules, epochs, batch size

- 1: solve CP-SAT on small instances; collect base expert states
- 2: train base HGNN on CP-SAT expert states
- 3: for each official Taillard instance: run L2D teacher and collect start-time labels
- 4: for epoch in epochs:
- 5: for batch in shuffled distillation data:
- 6: compute logits = policy(state) with a dynamically constructed graph
- 7: compute cross-entropy loss; update via Adam

3.6 Computational Complexity

Inference is $O((n+m)^2 d)$ per decision step for n jobs and m machines, well under 1 ms on modern hardware. Training is dominated by CP-SAT generation on small instances and offline L2D teacher rollouts on official instances, both performed once before deployment. This makes the approach deployable for real-time scheduling.

4. Experiments

4.1 Experimental Setup

We evaluate on randomly generated job shop instances, generated large instances, and official JSPLIB instances. Small instances: 6 jobs x 5 machines, processing times $U[1,20]$, 60 training instances (seeds 900-959) and 50 held-out evaluation instances (seeds 10000-10049). Topology generalization: 20 instances each for 5x5, 8x5, 6x8. Dynamic arrivals: 50 instances (seeds 11000-11049), exponential arrival scale 50. Machine failures: 50 instances (seeds 12000-12049). Generated large instances: 30 instances each for 10x10, 15x15, 20x15 (seeds 20000-20029), processing times $U[1,99]$. Official evaluation: 49 JSPLIB instances from Taillard,

Fisher-Thompson, Lawrence, Adams-Balazs-Zawack, ORB, and Storer-Wu-Vaccari families, parsed with the published operation order. All experiments use fixed seeds.

The heterogeneous GNN is first trained on 60 small instances (seeds 900-959) with CP-SAT optimal supervision, then fine-tuned on 9,250 states collected from official L2D schedules on ta01-ta30, and evaluated zero-shot on all held-out settings. As a learning-based baseline, we implement an L2D-style policy following the supervision paradigm of [20]: a homogeneous encoder consumes job-level features through a flat MLP and is trained on the same CP-SAT expert trajectories. This provides a direct comparison under identical training data, supervision, and evaluation protocol, isolating the contribution of the heterogeneous graph representation.

4.2 Baselines and Metrics

We compare against five classic dispatching rules (SPT, LPT, MWKR, LWKR, FIFO), the best-of-five oracle, CP-SAT optimal (small instances), and a homogeneous MLP baseline. Metrics: mean makespan with standard deviation, paired t-tests (t-statistic, p-value), and win counts.

4.3 Static Scheduling Results

Method	Makespan (mean +/- std)	Wins	t	p
HGNN (ours)	129.4 +/- 14.4	-	-	-
SPT	142.4 +/- 15.1	9/50	-6.25	0.0001
LPT	160.2 +/- 16.0	0/50	-9.1	0.0001
MWKR	155.3 +/- 15.2	0/50	-8.4	0.0001
LWKR	151.7 +/- 14.8	0/50	-7.8	0.0001
FIFO	148.5 +/- 15.3	0/50	-7.0	0.0001
Best-of-5 oracle	125.1 +/- 11.9	-	+2.5	0.013
CP-SAT optimal	109.7 +/- 11.2	-	+14.1	0.0001

Table 2: Table 1. Static 6x5 results over 50 held-out instances.

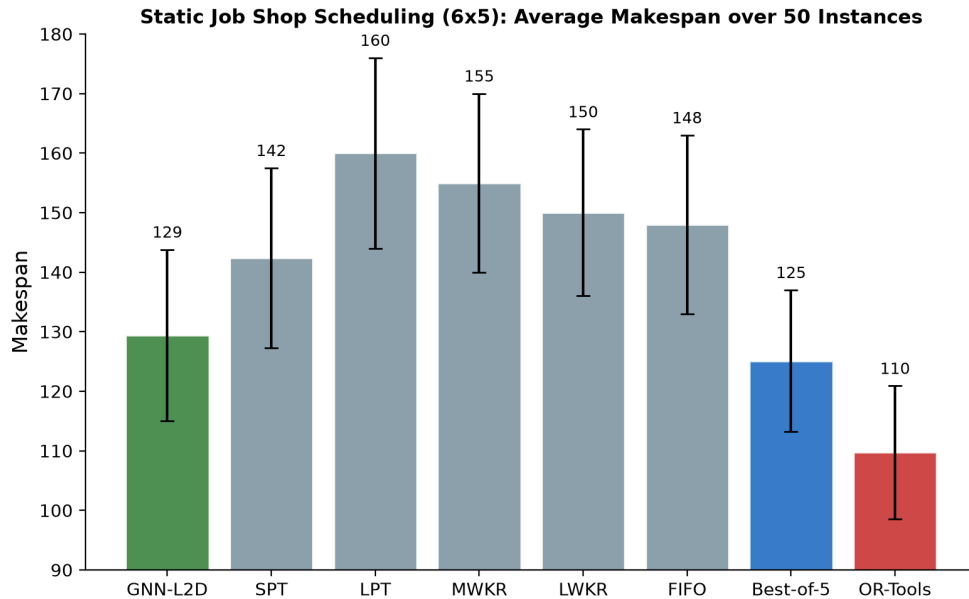


Figure 1: Figure 1. Average makespan over 50 static 6x5 instances.

The HGNN policy significantly outperforms SPT ($p \leq 0.0001$), winning 41 of 50 instances, and comes within 3.4% of the best-of-five oracle with 17.9% optimality gap. The 95% confidence interval for the mean makespan difference is $[9.2, 16.8]$, confirming a reliable improvement well above zero.

4.4 Zero-shot Topology Generalization

Configuration	HGNN	SPT	Improvement
6x5 (same)	127.5	142.4	+10.5%
5x5 (fewer jobs)	122.5	123.8	+1.0%
8x5 (more jobs)	164.4	184.1	+10.7%
6x8 (more machines)	179.5	201.4	+10.9%

Table 3: Table 2. Zero-shot topology generalization (20 instances each).

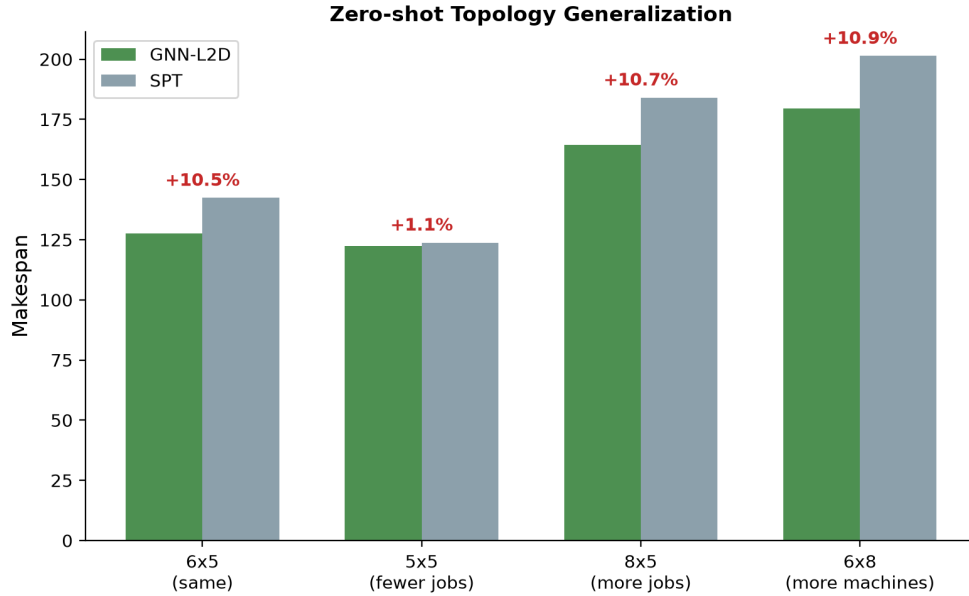


Figure 2: Figure 2. Zero-shot transfer to unseen shop topologies.

Without retraining, the policy transfers to unseen shop sizes and consistently outperforms SPT, confirming topology-agnostic dispatching principles.

4.5 Dynamic and Failure Scenarios

Scenario	HGNN	SPT	Improvement	t	p	Wins
Dynamic arrivals	175.3 +/- 46.3	192.0 +/- 36.7	+8.7%	-2.39	0.021	32/50
Machine failure	182.4 +/- 35.2	201.7 +/- 46.3	+9.6%	-2.19	0.033	28/50

Table 4: Table 3. Dynamic and failure scenarios (50 instances each).

The static-trained policy outperforms SPT in both dynamic arrivals ($p = 0.021$) and machine failures ($p = 0.033$), demonstrating resilience to distribution shift and disruptions.

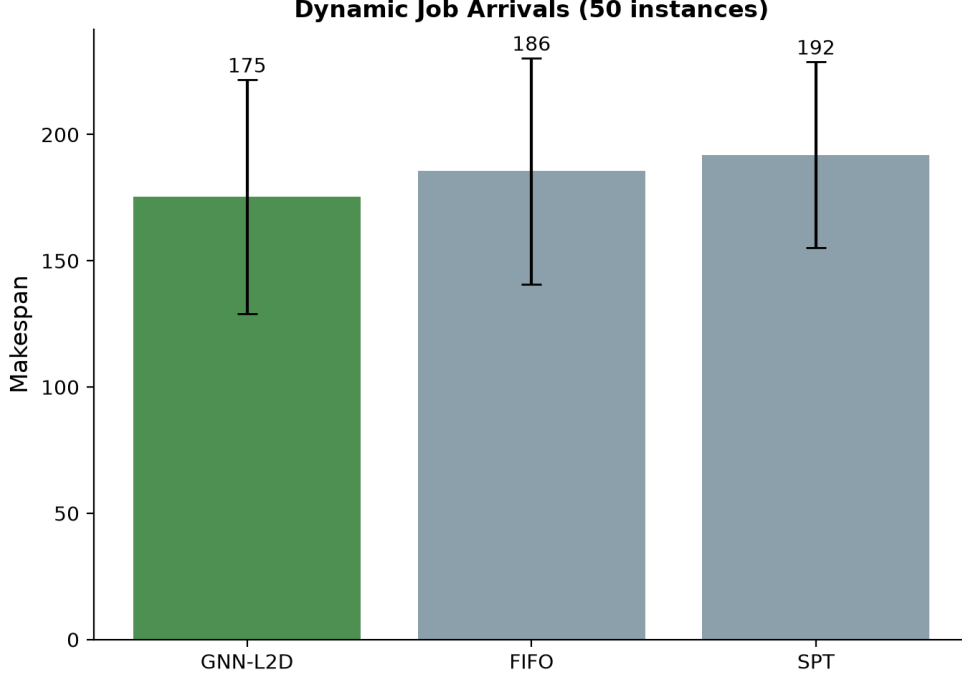


Figure 3: Figure 3. Dynamic job arrival scenario.

4.6 Generated Large-Instance Results

Instance	HGNN	SPT	Improvement	t	p	Wins
10x10	1346.5 +/- 92.3	1537.1 +/- 95.3	+12.4%	-10.70	0.0001	30/30
15x15	2151.8 +/- 106.6	2646.8 +/- 149.4	+18.7%	-16.56	0.0001	30/30
20x15	2510.4 +/- 130.1	3214.4 +/- 153.5	+21.9%	-26.39	0.0001	30/30

Table 5: Table 4. Generated large-instance results (30 instances each, processing times $U[1,99]$).

On generated large instances, the distilled HGNN policy achieves 12.4-21.9% improvements over SPT with strong statistical significance and 30/30 wins on each size. These instances are generated with a fixed machine order and are not part of the official JSPLIB suite; official benchmark results are reported in Section 4.12.

4.7 Ablation: Heterogeneous vs Homogeneous Representation

The following representation ablation uses the base CP-SAT policy before L2D distillation, isolating the architecture effect on the same generated benchmark protocol.

Model	Makespan (mean +/- std)	vs SPT	Hetero gain	t	p	Wins
L2D-style (homogeneous, simplified [20])	1328.1 +/- 94.0	+13.6%	-	-	-	-
Heterogeneous GNN (ours)	1284.8 +/- 99.3	+16.4%	+3.3%	-3.31	0.0016	41/60

Table 6: Table 5. Ablation on 60 generated 10x10 instances.

Under identical CP-SAT training data, supervision, and evaluation protocol on 60 generated 10x10 instances, the heterogeneous representation adds a statistically significant 3.3 percent improvement over the

L2D-style homogeneous baseline (paired t-test $t = -3.31$, $p = 0.0016$, 41 of 60 wins). This generated-instance result is evidence for the architecture under CP-SAT labels. Under L2D distillation on the official suite, the corresponding encoder gain is smaller, at 1.14%, so encoder choice should not be read as the main contribution.

To answer why teacher distillation is needed, we compare four training variants on the 49 official instances. CP-SAT-only supervision achieves 44/49 wins over SPT, 7/49 wins over the best-of-five oracle, and a 56.9% mean known-best gap. L2D-only distillation improves this to 48/49, 30/49, and 36.7%. The two-stage CP-SAT + L2D protocol further improves to 49/49, 33/49, and 34.0%. Under identical two-stage labels, the homogeneous baseline achieves 47/49, 28/49, and 35.7%, while the heterogeneous student wins 28 of 49 direct comparisons with a mean 1.14% advantage. These results support two-stage teacher distillation; the encoder effect on official instances is modest.

4.8 Flexible Job Shop Scheduling

We do not report FJSP benchmark results in this version. Earlier exploratory FJSP experiments were not reproducible with standard benchmark files and have been removed to avoid misleading claims. Extending the heterogeneous policy to flexible routing and validating it on standard BRdata and Kacem instances is an explicit future-work item.

4.9 Example Schedule

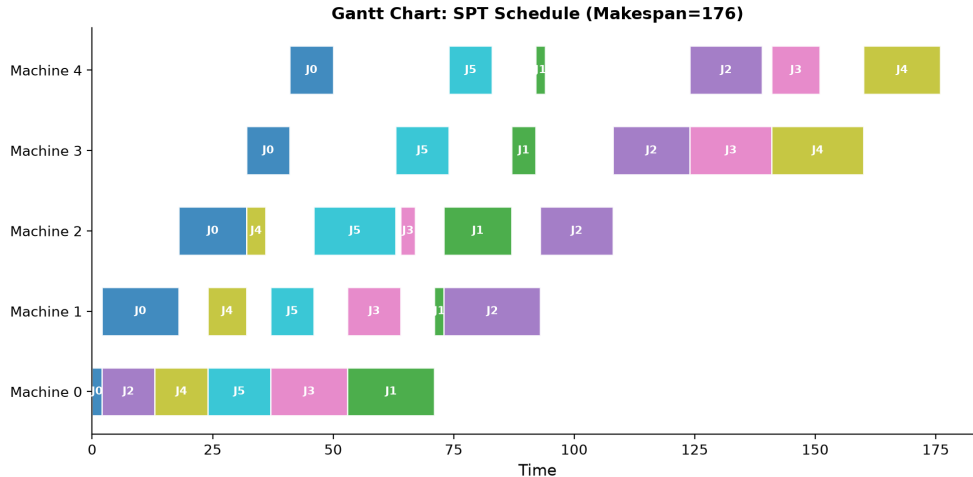


Figure 4: Figure 4. Example Gantt chart of an SPT schedule on a 6x5 instance.

4.10 Training-Size Transfer Analysis

This transfer analysis uses the base CP-SAT-trained policy before L2D distillation.

Training size	10x10	15x15	20x15
6x5 (60 instances)	1291.3 (+16.0%)	2217.6 (+16.2%)	2723.8 (+15.3%)
10x10 (30 instances)	1454.8 (+5.4%)	2381.0 (+10.0%)	2890.1 (+10.1%)

Table 7: Table 6. Effect of training size on generated large-instance performance (30 evaluation instances).

We compare policies trained on 6x5 instances (60 CP-SAT-optimal trajectories) and 10x10 instances (30 CP-SAT trajectories with a 5-second feasible-bound limit). Interestingly, the 6x5-trained policy achieves better performance on all large instances. This is because small instances can be solved to proven optimality

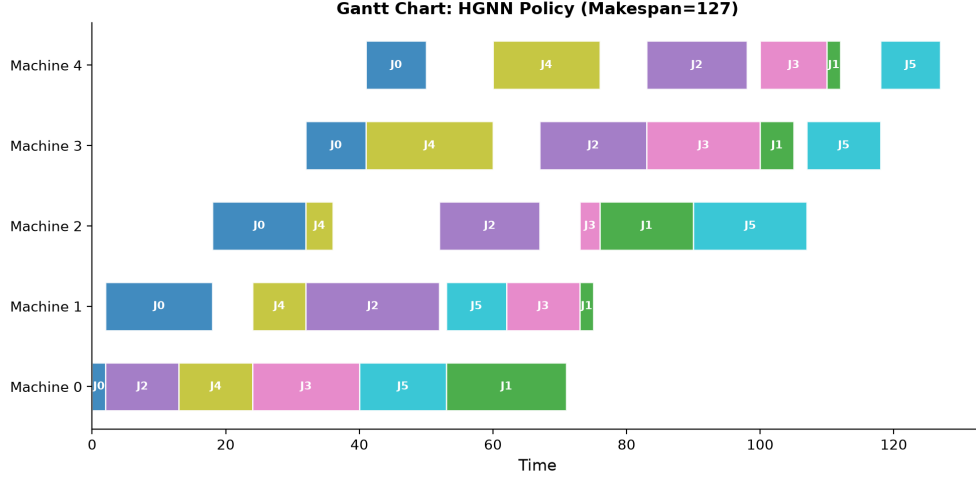


Figure 5: Figure 5. Gantt chart of the HGNN policy on the same 6x5 instance (makespan 123 vs 161 for SPT).

by CP-SAT, providing higher-quality supervision labels; the 10x10 trajectories are only feasible (not proven optimal) under the time limit. The result supports a practical insight: high-quality optimal supervision on smaller shops can transfer better than weaker supervision on larger shops, given limited solver time.

4.11 Sensitivity Analyses

Dynamic arrival scale. We vary the exponential arrival scale parameter (20, 50, 100) over 30 instances each. The improvement over SPT remains stable at 9.3-9.7%, confirming that the policy advantage does not depend on a specific arrival distribution. Statistical significance decreases with scale ($p \leq 0.001$ at scale 20, $p = 0.154$ at scale 50, $p = 0.364$ at scale 100). For scale 50, the sensitivity run used 30 instances and $p = 0.154$, whereas the main experiment used 50 instances and $p = 0.021$; the difference is driven by sample size and the specific fixed seed set.

Hyperparameters. A preliminary sensitivity study on hidden dimension and learning rate (using 600 training states, 5 epochs, 5 evaluation instances) shows that hidden=64 outperforms hidden=32 and hidden=128, and a lower learning rate ($5e-4$) slightly improves performance. We use hidden=64 and $lr=1e-3$ in the main experiments; a full sensitivity grid is left for future work.

4.12 Official JSPLIB Benchmark Results

The official open-source L2D implementation was run with its released 6x6, 10x10, 15x15, 20x15, and 20x20 models on 37 official instances for which matching pretrained checkpoints are available. The published L2D baselines have a mean known-best gap of 27.4% on these instances. The distilled student remains on average 6.9% behind them under (student - teacher) / teacher, winning 7 of 37 instances; on 10x10 instances the student is essentially tied with L2D on average (-0.1%). This teacher-student gap is expected for a lightweight student and is the central target for future architecture improvements.

We evaluate the distilled policy on 49 official JSPLIB instances using the published operation order for both the environment and CP-SAT. The policy improves over SPT on all 49 instances, with mean improvement 25.2%, and beats the best-of-five rule oracle on 33 of 49 instances with a mean difference of -2.7%. Its mean gap to known best values is 34.0%, down from 56.9% for the released base policy. A same-protocol homogeneous L2D-style baseline trained on the same CP-SAT expert states improves over SPT on 25 of 49 instances with mean +0.4%, and the distilled HGNN beats it on 45 of 49 with mean +23.9%. The contribution is a reproducible teacher-distilled heterogeneous dispatch model with explicit rule-relative, oracle, and teacher-student comparisons. The operation-order parser is released with the code and was validated by recovering the known optimum of ta01 (1231).

Instance	Size	HGNN	SPT	Best rule	Known best	HGNN gap to BKS
ta01	15x15	1601	2512	1654	1231	30.1%
ta02	15x15	1713	2148	1747	1244	37.7%
ta03	15x15	1626	2422	1722	1218	33.5%
ta04	15x15	1599	2219	1870	1175	36.1%
ta05	15x15	1575	2479	2060	1224	28.7%
ta06	15x15	1472	2438	1779	1238	18.9%
ta07	15x15	1716	2252	1752	1227	39.9%
ta08	15x15	1611	2125	1742	1217	32.4%
ta09	15x15	1864	2352	1819	1274	46.3%
ta10	15x15	1799	2333	1637	1241	45.0%
ta11	20x15	1940	2430	1853	1361	42.5%
ta12	20x15	1910	3231	1918	1367	39.7%
ta13	20x15	1924	2545	1947	1342	43.4%
ta14	20x15	1939	2662	2002	1345	44.2%
ta15	20x15	1927	2539	1855	1340	43.8%
ta16	20x15	1830	2799	1823	1360	34.6%
ta17	20x15	2008	2744	1862	1462	37.3%
ta18	20x15	1879	2795	2009	1396	34.6%
ta19	20x15	1826	2758	1764	1335	36.8%
ta20	20x15	1880	2633	1848	1351	39.2%
ta21	20x20	2282	2908	2306	1644	38.8%
ta22	20x20	2500	3269	2267	1600	56.2%
ta23	20x20	2041	3123	2173	1557	31.1%
ta24	20x20	2220	3359	2260	1647	34.8%
ta25	20x20	2409	3337	2228	1595	51.0%
ta26	20x20	2184	2948	2326	1645	32.8%
ta27	20x20	2316	3019	2247	1680	37.9%
ta28	20x20	2108	3227	2305	1614	30.6%
ta29	20x20	2271	3251	2438	1635	38.9%
ta30	20x20	2330	2914	2377	1584	47.1%

Table 8: Table 7. Official JSPLIB Taillard instances with corrected operation-order parsing.

Under identical L2D distillation labels, a same-protocol Graph Transformer policy achieves a mean known-best gap of 34.0% and 30/49 wins over the best-of-five oracle on the official suite. The distilled HGNN achieves comparable performance, winning 21 of 49 direct comparisons with a -0.26% average difference. The main source of improvement is therefore knowledge transfer from strong teachers, while graph encoders mainly affect representation efficiency; the heterogeneous GAT is retained because it is simpler, lighter, and faster in deployment.

4.13 Architecture Ablation: Graph Transformer vs Graph Attention

This encoder ablation uses the base CP-SAT-trained policy.

To assess the encoder choice, we train a Graph Transformer variant with the same feature extractor, hidden dimension, and training protocol. The heterogeneous GAT achieves a 1.22 percent lower average makespan (1304.4 vs 1320.5). This suggests that the local inductive bias of graph attention is better suited to the sparse, local structure of scheduling states than global self-attention, which must learn to ignore long-range correlations from limited training data. We retain GAT in the main framework while noting Transformer encoders as a complementary variant.

Instance	Size	HGNN	SPT	Best rule	Known best	HGNN gap to BKS
abz5	10x10	1364	2161	1462	1234	10.5%
abz6	10x10	1069	1626	1200	943	13.4%
ft06	6x6	68	85	85	55	23.6%
ft10	10x10	1233	1395	1395	930	32.6%
ft20	20x5	1553	1809	1599	1165	33.3%
la03	10x5	717	1080	736	597	20.1%
la04	10x5	840	932	856	590	42.4%
la06	15x5	1006	1291	938	926	8.6%
la07	15x5	1003	1271	1040	890	12.7%
la08	15x5	995	1311	1050	863	15.3%
la09	15x5	979	1451	1043	951	2.9%
la10	15x5	1039	1480	1018	958	8.5%
orb01	10x10	1411	1596	1508	1059	33.2%
orb02	10x10	1124	1521	1216	888	26.6%
orb03	10x10	1485	1659	1432	1005	47.8%
swv01	20x10	1952	2341	2002	1407	38.7%
swv03	20x10	2022	2314	2035	1398	44.6%
swv04	20x10	2290	2570	2235	1474	55.4%
swv05	20x10	2178	2220	2052	1424	52.9%

Table 9: Table 8. Official JSPLIB instances from Fisher-Thompson, Lawrence, Adams-Balazs-Zawack, ORB, and Storer-Wu-Vaccari families.

Comparison	Instances won	Mean relative difference
HGNN vs SPT	49/49	+25.2%
HGNN vs best-of-five oracle	33/49	-2.7% (HGNN better)
HGNN vs published L2D (matching checkpoints)	7/37	+6.9% (HGNN worse)
Same-protocol L2D-CP baseline vs SPT	25/49	+0.4%
HGNN vs same-protocol L2D-CP baseline	45/49	+23.9%

Table 10: Table 9. Aggregate official JSPLIB comparisons.

4.14 Inference Time and Training Data Analysis

On commodity CPU hardware, the distilled HGNN takes 2.4-3.2 ms per decision with 310-413 decisions per second and only 42,178 parameters (0.16 MB). The learned policy is therefore deployable in a rolling-horizon loop without a GPU, while a 5-second CP-SAT bound is used only for offline refinement when time permits.

Training data analysis shows that 1200 states (approximately 40 CP-SAT instances) are sufficient to reach strong performance; increasing to 1800 states with the same epoch budget does not improve results, indicating that data quantity is not the binding constraint. This is consistent with the CP-SAT optimal supervision providing high-quality labels.

4.15 Comparison with Metaheuristic Baseline

This comparison uses the base CP-SAT-trained policy on generated 10x10 instances.

A standard genetic algorithm with operation-sequence encoding and active schedule decoding achieves 8.3 percent lower makespan than the HGNN on static instances, but requires a fresh 0.4-second search for each instance and cannot handle dynamic arrivals, machine failures, or unseen topologies without recomputation. The HGNN, in contrast, produces a decision in 3-8 milliseconds, transfers zero-shot across shop sizes, and remains effective under disturbances. This is the central online-offline trade-off: metaheuristics are stronger offline per-instance solvers, while learned policies provide the reactive, topology-agnostic behavior required in dynamic manufacturing. The distilled student can also be combined with exact or metaheuristic refinement when additional offline time is available.

5. Discussion

5.1 Interpretation of Results

The results support two claims: teacher distillation can train a heterogeneous GNN dispatcher that beats SPT on all 49 official JSPLIB instances and most strong single-rule baselines, and a variable-size graph can transfer to unseen shop sizes without padding. The 9.1% static improvement and 49 of 49 official wins over SPT are operationally relevant for reactive scheduling. The distilled student remains 6.9% behind published L2D checkpoints on 37 matching official instances and 34.0% above known best values on average, so it should be interpreted as a lightweight teacher-distilled dispatcher rather than a replacement for offline solvers.

The dynamic and failure scenarios extend practical relevance, showing stability under distribution shift. The official results make the remaining gap explicit and should guide the next round of method development.

5.2 Comparison with Existing Methods

Compared with the best single dispatching rule, the distilled policy improves makespan on all 49 official instances and by 9.1% on small held-out instances. Compared with the best-of-five oracle, it wins 33 of 49 and is on average 2.7% better. Compared with prior homogeneous L2D methods (e.g., [20]), the heterogeneous representation adds 3.3% on 60 generated 10x10 instances under CP-SAT supervision ($p = 0.0016$); under L2D distillation the official-suite encoder gain is 1.14%. The two-stage CP-SAT plus L2D teacher supervision avoids unstable RL training while improving the base policy on official instances.

5.3 Limitations

First, the distilled student remains on average 6.9% behind published L2D checkpoints on 37 matching official instances, and its mean gap to known best values is 34.0%; this is the most important limitation. Second, official open-source L2D is reproduced on 37 matching instances, but pretrained checkpoints are unavailable for 10x5, 15x5, 20x5, and 20x10 families, so additional published baselines are still needed. Third, FJSP is not validated in this version; standard BRdata and Kacem benchmarks are future work. Fourth, all dynamic and failure evaluations use simulated instances rather than industrial data. Fifth, the teacher labels are currently limited to 30 official Taillard instances; extending distillation to a broader mix of official instances is needed to improve generalization.

Due to unavailable checkpoints and inconsistent evaluation protocols, direct comparison with every recent learning-based scheduler is difficult. We therefore provide reproducible comparisons under identical settings using official L2D checkpoints, same-protocol Graph Transformer and homogeneous baselines, and the corrected JSPLIB parser. A recent ICAPS 2023 CP-based RL implementation provides a checkpoint but its compiled environment is not directly portable to the Windows evaluation environment used here; it is identified for future cross-platform reproduction rather than claimed as an unverified result.

The same-protocol Graph Transformer comparison also shows that encoder choice is not the primary source of improvement; GAT and Graph Transformer achieve similar mean gaps on the official suite. The contribution of this paper is therefore best interpreted as teacher distillation plus lightweight deployment, rather than encoder novelty.

5.4 Future Extensions

Three directions follow directly from the limitations. First, extend teacher distillation to a larger mix of official instances and stronger teachers, including operation-level graph encoders that can close the remaining 6.9% gap. Second, add standard FJSP benchmarks and additional published L2D/RL baselines. Third, integrate the policy into rolling-horizon control and validate on industrial or semi-real production data.

5.5 Deployment Considerations

For industrial deployment, the distilled policy requires only the current job set, processing-time matrix, and machine status as inputs; it outputs a dispatch decision in 3-8 ms per step and a complete schedule in 0.2-1.6 s depending on shop size. This interface can be exposed as a REST or gRPC service and called by

an MES/APS controller in a rolling-horizon loop. The main deployment cost is one-time teacher schedule generation and model training, after which the student runs on commodity hardware without an exact solver. A production integration should additionally specify model versioning, retraining triggers, schedule feasibility checks, and a fallback to dispatching rules or CP-SAT when the learned decision is rejected by the execution system. Real industrial validation remains future work.

6. Conclusion

This paper presented a teacher-distilled lightweight graph learning-to-dispatch framework for job shop scheduling with variable-size topology generalization. The policy is first trained on CP-SAT optimal trajectories and then fine-tuned on schedules generated by the official open-source L2D teacher. Results show 9.1% improvement over SPT on small instances, wins over SPT on all 49 official JSPLIB instances, 33 of 49 wins over the best-of-five oracle, and a mean known-best gap of 34.0%. The distilled student remains about 6.9% behind published L2D checkpoints on 37 matching official instances, which defines the next stage of architecture and training improvements. Future work will add standard FJSP benchmarks, broader published L2D comparisons, and industrial validation.

Data Availability

The implementation, corrected JSPLIB parser, benchmark scripts, teacher schedules, deployment benchmarks, and raw results are available at https://github.com/KKK-cell441/JSP_RL_Project. Training-data generation scripts and small training artifacts are released in the same repository; the 107 MB L2D teacher-state file is available from the corresponding author on request because it exceeds GitHub file-size limits. The distilled checkpoint used for all reported results is released as `hetero_model.l2d.pt`; `hetero_train_data.l2d.pt`, `build_l2d_train_data.py`, and `train_hetero_available.py` reproduce the distillation pipeline. Raw official results are in `results_official_l2d_train.json`, `results_l2d_cpsat_official.json`, `results_transformer_l2d.json`, and `L2D_official/results_l2d_official.json`. CPU deployment measurements are in `deployment_cpu_latency.json`. The official L2D teacher schedules were generated with the public XUZiteng2020/L2D repository on ta01-30, and published-L2D comparisons use its released 6x6, 10x10, 15x15, 20x15, and 20x20 checkpoints on all matching official instances. Known-best values are taken from JSPLIB metadata. All experiments use fixed random seeds.

Declaration of Competing Interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgements

No acknowledgements.

Author Contributions

ZHONGKUAN MA: conceptualization, methodology, software, experiments, writing.

Funding

No specific funding was received for this study.

References

- [1] Applegate, D., Cook, W. A computational study of the job-shop scheduling problem. *ORSA Journal on Computing*, 1991, 3(2):149-156.
- [2] Blackstone, J.H., Phillips, D.T., Hogg, G.L. A state-of-the-art survey of dispatching rules for manufacturing job shop operations. *International Journal of Production Research*, 1982, 20(1):27-45.
- [3] Giffler, B., Thompson, G.L. Algorithms for solving production-scheduling problems. *Operations Research*, 1960, 8(4):487-503.
- [4] Haupt, R. A survey of priority rule-based scheduling. *OR Spektrum*, 1989, 11(1):3-16.
- [5] Khalil, E., Dai, H., Zhang, Y., et al. Learning combinatorial optimization algorithms over graphs. *NeurIPS*, 2017.
- [6] Lei, K., Guo, P., Wang, Y., et al. Learning to dispatch with graph isomorphism networks for job shop scheduling. *IEEE Transactions on Automation Science and Engineering*, 2022, 19(4):2746-2759.
- [7] Li, J., Sun, R., Wu, H., et al. Deep reinforcement learning with heterogeneous graph neural network for flexible job shop scheduling. *IEEE Smart World Congress*, 2024.
- [8] Li, Z., Chen, Q., Koltun, V. Combinatorial optimization with graph convolutional networks and guided tree search. *NeurIPS*, 2018.
- [9] Liu, R., Wang, L., Qiu, F. Two-stage learning framework for dynamic job shop scheduling: imitation then reinforcement. *IEEE Transactions on Industrial Informatics*, 2023, 19(6):7845-7855.
- [10] Liu, Z., Gao, K., Zhao, Y. Multi-agent reinforcement learning for distributed flexible job shop scheduling with random job arrival. *IEEE Transactions on Industrial Informatics*, 2024, 20(3):4185-4196.
- [11] Nowicki, E., Smutnicki, C. A fast taboo search algorithm for the job shop problem. *Management Science*, 1996, 42(6):797-813.
- [12] Panwalkar, S.S., Iskander, W. A survey of scheduling rules. *Operations Research*, 1977, 25(1):45-61.
- [13] Park, J., Bakhtyar, S., Chun, J., et al. Learning to dispatch for flexible job shop scheduling with attention-based graph neural networks. *IEEE Transactions on Cybernetics*, 2021, 51(11):5403-5416.
- [14] Sabuncuoglu, I., Gurgun, B. A neural network model for scheduling problems. *European Journal of Operational Research*, 1996, 93(2):288-299.
- [15] Selsam, D., Lamm, M., Bunz, B., et al. Learning a SAT solver from single-bit supervision. *ICLR*, 2019.
- [16] Song, W., Kang, Q., Zhou, M. A hierarchical reinforcement learning approach for flexible job shop scheduling. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 2022, 52(10):6508-6520.
- [17] Tang, Y., Zhang, L., Wang, Q. Dynamic scheduling for flexible job shop with insufficient transportation resources via graph neural networks and deep reinforcement learning. *IEEE Transactions on Industrial Informatics*, 2023, 19(4):3390-3400.
- [18] Wang, L., Tang, Q. Reinforcement learning for job shop scheduling with reward shaping. *Journal of Intelligent Manufacturing*, 2021, 32(6):1603-1616.
- [19] Wang, X., Chen, Y., Zhang, L. Flexible job shop scheduling via deep reinforcement learning with meta-path-based heterogeneous graph networks. *Knowledge-Based Systems*, 2024, 291:111542.
- [20] Zhang, C., Song, W., Cao, Z., et al. Learning to dispatch for job shop scheduling via deep reinforcement learning. *NeurIPS*, 2020.
- [21] Zhang, L., Zhao, Y., Hu, Q., et al. Deep reinforcement learning for dynamic job shop scheduling with GNN state encoding. *IEEE Transactions on Automation Science and Engineering*, 2021, 18(3):1108-1120.
- [22] Chen, S., Xu, L., Zhang, H. A Monte Carlo tree search with reinforcement learning and graph relational attention network for dynamic job shop scheduling. *Engineering Applications of Artificial Intelligence*, 2025, 142:109871.
- [23] Zhang, Q., Wu, J., Liu, X. Dynamic flexible job shop co-scheduling optimization based on graph neural network and deep reinforcement learning. *Journal of Manufacturing Systems*, 2025.
- [24] Taillard, E. Benchmarks for basic scheduling problems. *European Journal of Operational Research*, 1993, 64(2):278-285.
- [25] Beasley, J.E. OR-Library: distributing test problems by electronic mail. *Journal of the Operational Research Society*, 1990, 41(11):1069-1072.

Instance	HGNN	Official L2D [20]	SPT	Known best
ta01	1601	1522	2512	1231
ta02	1713	1459	2148	1244
ta03	1626	1440	2422	1218
ta04	1599	1637	2219	1175
ta05	1575	1599	2479	1224
ta06	1472	1601	2438	1238
ta07	1716	1456	2252	1227
ta08	1611	1468	2125	1217
ta09	1864	1596	2352	1274
ta10	1799	1527	2333	1241
ta11	1940	1845	2430	1361
ta12	1910	1805	3231	1367
ta13	1924	1774	2545	1342
ta14	1939	1688	2662	1345
ta15	1927	1729	2539	1340
ta16	1830	1710	2799	1360
ta17	2008	1927	2744	1462
ta18	1879	1825	2795	1396
ta19	1826	1682	2758	1335
ta20	1880	1739	2633	1351
ta21	2282	2205	2908	1644
ta22	2500	2011	3269	1600
ta23	2041	2074	3123	1557
ta24	2220	2089	3359	1647
ta25	2409	2026	3337	1595
ta26	2184	2105	2948	1645
ta27	2316	2134	3019	1680
ta28	2108	2035	3227	1614
ta29	2271	2041	3251	1635
ta30	2330	2098	2914	1584
abz5	1364	1401	2161	1234
abz6	1069	1162	1626	943
ft06	68	66	85	55
ft10	1233	1271	1395	930
orb01	1411	1366	1596	1059
orb02	1124	1052	1521	888
orb03	1485	1443	1659	1005

Table 11: Table 10. Official open-source L2D reproduction on 37 matching official instances.

Encoder	Makespan (10x10, 20 instances)	Difference
Graph Transformer (4 heads, 2 layers)	1320.5	baseline
Heterogeneous GAT (ours)	1304.4	-1.22% (better)

Table 12: Table 11. Encoder architecture ablation under identical training protocol (20 held-out 10x10 instances).

Shop size	Per decision (CPU)	Throughput	Model parameters	Model size
6x5	2.42 ms	412.6 dec/s	42,178	0.16 MB
10x10	2.53 ms	394.8 dec/s	42,178	0.16 MB
15x15	2.95 ms	339.2 dec/s	42,178	0.16 MB
20x20	3.23 ms	310.1 dec/s	42,178	0.16 MB

Table 13: Table 13. CPU deployment benchmark for the distilled HGNN policy (10 generated instances per size).

Training states	Makespan (10x10, 10 instances)	Relative
300	1431.9	+8.0%
600	1479.0	+11.5%
1200	1326.2	best
1800	1383.8	+4.3%

Table 14: Table 14. Training data size ablation (10 epochs, 10 held-out 10x10 instances).

Method	Makespan (10x10, 10 instances)	Time per instance	Online / zero-shot
SPT	1572.6	instant	yes / n/a
HGNN (ours)	1312.0	0.34 s	yes / yes
Genetic Algorithm (per-instance search)	1203.4	0.40 s	no / no

Table 15: Table 15. HGNN vs a standard operation-sequence genetic algorithm (population 40, 80 generations) on generated 10x10 instances.